

P1029R2: move = relocates

Document #: P1029R2
Date: 2019-06-14
Project: Programming Language C++
Evolution Working Group
Reply-to: Niall Douglas
<s_sourceforge@nedprod.com>

This proposes a new default for C++ move constructors = **relocates**, which enables more aggressive optimisation of move constructions for such types than is possible at present. The R0 edition of this paper received the following vote at the May 2018 meeting of SG14: 1/10/2/0/0 (SF/WF/N/WA/SA), however this proposal has evolved significantly since then.

The primary motivation of this proposal is to propose a form of move relocation so unambitious, uncontentious and conservative that it has a realistic chance of getting approved by WG21. Other proposals e.g [P1144] *Object relocation in terms of move plus destroy*, or [P1631] *Object detachment and attachment* are more ambitious.

Something similar in effect, though not in semantics, to this proposed feature is already in the clang compiler via the `[[clang::trivial_abi]]` attribute¹. The main difference is that this proposal also supports move relocating polymorphic types.

Changes since R1:

- Taking in feedback from R1, replaced the `[[move_relocates]]` attribute nomenclature with = **relocates**, as this proposal involves an ABI break for move relocatable types in newer compilers. The ABI break implies that an attribute is the wrong nomenclature.

Contents

1	Introduction	2
1.1	Other work in this area	4
2	Impact on the Standard	4
3	Proposed Design	5
3.1	Worked example, and effect on codegen	6
3.1.1	With current compilers, without = relocates :	7
3.1.2	With the proposed = relocates :	9
3.1.3	How do you know that the code in the second example is feasibly generatable by a compiler?	9
3.2	So what?	10

¹<https://clang.llvm.org/docs/AttributeReference.html#trivial-abi-clang-trivial-abi>

4	Design decisions, guidelines and rationale	10
5	Technical specifications	10
6	Acknowledgements	10
7	References	11

1 Introduction

The most aggressive optimisations which the C++ compiler can currently perform are to types which meet the `TriviallyCopyable` requirements:

- Every copy constructor is trivial or deleted.
- Every move constructor is trivial or deleted.
- Every copy assignment operator is trivial or deleted.
- Every move assignment operator is trivial or deleted.
- At least one copy constructor, move constructor, copy assignment operator, or move assignment operator is non-deleted.
- Trivial non-deleted destructor.

All the integral types meet `TriviallyCopyable`, as do C structures. The compiler is thus free to store such types in CPU registers, relocate them at its convenience in memory as if by `memcpy`, and overwrite their storage as no destruction is needed. This greatly simplifies the job of the compiler optimiser, making for tighter codegen, faster compile times, and less stack usage, all highly desirable things.

There are quite a lot of types in the standard library and in user code which do not meet `TriviallyCopyable`, yet are completely safe to be relocated arbitrarily, at any time and for any reason, in memory as if by `memcpy`. For example, a `std::unique_ptr<T>` implementation might have a similar implementation to:

```

1  template<class T> class unique_ptr
2  {
3      T *_ptr{nullptr};
4  public:
5      unique_ptr() = default;
6      unique_ptr(unique_ptr &&o) : _ptr(o._ptr) { o._ptr = nullptr; }
7      ~unique_ptr() { delete _ptr; _ptr = nullptr; }
8      ...
9  };

```

In current compilers, returning from a function a `std::unique_ptr<T>` will have a much heavier ABI overhead over returning a `T*`, because the lack of trivial copyability means that the compiler must use the stack to return unique ptrs, whereas the naked pointer can be directly returned in a CPU register (see worked example in assembler later in this paper).

With `= relocates`, a `unique_ptr` implementation might instead be written:

```
1 template<class T> class unique_ptr
2 {
3     T *_ptr{nullptr};
4 public:
5     constexpr unique_ptr() = default;
6     unique_ptr(unique_ptr &&) = relocates;           // This type is move relocatable!
7     ~unique_ptr() { delete _ptr; _ptr = nullptr; }
8     ...
9 };
```

... and the compiler would now know that this type can be arbitrarily relocated in memory, with no ill effect, via the following as-if sequence:

```
1 unique_ptr<T> *dest, *src;
2
3 // Copy bytes of src to dest
4 memcpy(dest, src, sizeof(unique_ptr<T>));
5
6 // Copy bytes of constexpr default constructed instance to src
7 unique_ptr<T> default_constructed;
8 memcpy(src, &default_constructed, sizeof(unique_ptr<T>));
```

When the move constructor `= relocates`, the programmer is giving the *explicit guarantee* to the compiler that for this type:

1. Move construction equals two as-if `memcpy()`'s, one from old storage to new, one from a **constexpr**² default constructed instance to old.
2. That any non-trivial destruction of a default constructed instance of the type *has no side effects* (and thus can be safely elided).

Because of these warranties made by the programmer to the C++ compiler, returning STL containers by value from functions can now be optimal in terms of codegen (see worked example in assembler later in this paper). A `std::vector<T>` with default allocator might have a similar implementation to:

```
1 template<class T> class vector
2 {
3     T *_begin{nullptr}, *_end{nullptr}, *_capacity{nullptr};
4 public:
5     constexpr vector() = default;
6     vector(vector &&) = relocates;
7     // memcpy src to dest, then memcpy constexpr default instance over src i.e. same as:
8     // vector(vector &&o) : _begin(o._begin), _end(o._end), _capacity(o._capacity) { o._begin = o._end =
9         o._capacity = nullptr; }
10     ~vector() { delete _begin; _begin = _end = _capacity = nullptr; }
11     ...
12 };
```

²In this proposal, one may only use `= relocates` with types with a in-class defined, constexpr default constructor.

Because the compiler knows that this type is move bitcopyable, and destructing moved-from instances has no side effects, it can bit copy the instance into CPU registers for the return rather than using the stack, if the target architecture has sufficient CPU registers return capacity. This brings the same power of optimisation to a large subset of non-trivially-copyable C++ types.

1.1 Other work in this area

This paper does NOT propose destructive moves. Object lifetimes are unchanged.

- [N4034] *Destructive Move*

This proposal differs from destructive moves in the following ways:

- This single purpose proposal only affects the strength of the guarantees provided by the programmer regarding the move constructor. It does not change what move construction means. It does not change object lifetimes.

- [P0023] *Relocator: Efficiently moving objects.*

This proposal differs from relocators in the following ways:

- We do not propose any new kind of operation, nor new operators. We propose stronger guarantees given by the programmer to the compiler for the implementation of move construction.

- [P1144] *Object relocation in terms of move plus destroy.*

This proposal differs from P1144 in the following ways:

- We do not propose any standard library support, new algorithms, new operations, nor distinguish between trivial and non-trivial relocatability. We simply propose being able to tell the compiler that move construction relocates the object.

- [P1631] *Object detachment and attachment.*

If this proposal is a single fix for a single problem, P1631 could be the basis for changing the C++ abstract machine to enable object relocation become completely natural, no special-casing nor fiddling with rules needed.

There is also the `[[clang::trivial_abi]]` attribute in the clang compiler which overrides the non-trivial treatment of a type for the purposes of moves and copies. This proposal affects move constructions only, but has a very similar positive effect on code generation.

2 Impact on the Standard

Very limited. This is a limited, opt-in, optimisation of the implementation of move construction only where the compiler can make some stronger assumptions during optimisation than it can ordinarily. We do not fiddle with allocators, the meaning nor semantics of moves, object lifetimes, destructors,

library code, nor anything else. We do not add the concept of relocatability to the standard, or change anything at all about lifetime or the object model.

3 Proposed Design

1. That a new default choice = **relocates** become applicable to move constructors. The programmer applies this default if they wish to guarantee to the compiler that the move constructor and destructor implementation have stronger guarantees than usual.
2. It shall be a compile time diagnostic if:
 - Not all base classes are either trivially copyable, or there is a move constructor in a base class not marked = **relocates**.
 - If there is a virtual inheritance anywhere in the inheritance tree.
 - Not all member variable data types are either trivially copyable, or any member data type has a move constructor not marked = **relocates**.
 - The type does not have a non-deleted, constexpr, in-class defined default constructor. This implies that all base classes and member variables must have a constexpr, in-class defined constructor. Note that the default constructor need not be public, = **relocates** is a move constructor implementation, and thus can call non-public member functions.
3. Types with virtual destructors and = **relocates** move constructors are permitted. If the programmer moves a type which has been inherited from, and its destructor reimplemented to have side effects when called on a default constructed instance, then that is a guarantee violation by the programmer.

[*Note:* This has obvious risks. Imagine someone inheriting from `std::exception` who replaces the destructor to do something important, and if the standard library then changes `std::exception` into a move relocating type. Whilst everything would probably ‘still work’, as the destructor is always called on the final instance of the object, there would be a loss of defined behaviour, as the custom destructor implementation which previously was called on moved-from instances, may no longer be so called.

The temptation will thus be to remove support for objects with virtual destructors entirely. However, I would argue that one of the most powerful motivating cases for move relocating types is for types with costly destructors. If we can find a way to retain support for those, I think we should³. – end note]

4. If a type `T`’s move constructor has been defaulted to = **relocates**, the compiler will implement the move constructor with an as-if `memcpy(&dest, &src, sizeof(T))`, followed by as-if

³At some future point, I will get around to writing up a proposal for the `[[no_side_effects]]` and `[[no_visible_side_effects]]` C++ attributes as used throughout P1031 *Low level file i/o*. These would be powerful enough to declare that any member function, when called with a default constructed instance, has no side effects, which is a generalisation of the above. What I do not know yet is whether a compiler can reliably detect when a virtual destructor has side effects when called on a default constructed instance, and thus issue a diagnostic.

`memcpy(&src, &T{}, sizeof(T))`. Note that by ‘as-if’, we mean that the compiler can fully optimise the sequence, including the elision of the second memory copy. The destructor is not ordinarily called on the source object, as the programmer has guaranteed that doing so on a default constructed instance has no side effects.

5. If a type `T`’s move constructor has been defaulted to `= relocates`, the trait `std::is_move_constructor_relocating<T>` shall be true.
6. Finally, the C++ standardese for this proposal would guarantee that move relocating types can *pass through* code with a C ABI. In other words, it would be defined behaviour for a move relocating type to be passed to an `extern "C"` function by value, under C2x’s compatible type rules. If the C ABI function is implemented in C, it would bit copy the value, and having no knowledge of destructors, it would not call destructors on copied-from instances. It would be undefined behaviour to send a move relocating object into C, and for that object instance to not eventually return to C++.

This may seem superfluous, but it would be a great boon to aid interoperation with other languages, which speak C. Right now, efficient other-language bindings generally must do lots of UB to avoid excessive memory copying and dynamic memory allocation for type erasure. If C++ could legally send a richer subset of C++ object types to C, particularly the use case of rich C++ types *passing through* C code e.g. via a C callback function, that would be very useful in removing the need for much UB in language interop, and enable a much larger subset of C++ to be directly invocable by C code.

3.1 Worked example, and effect on codegen

Let us take a worked example. Imagine the following partial implementation of `unique_ptr`:

```

1  template<class T>
2  class unique_ptr
3  {
4      T *_v{nullptr};
5  public:
6      // Has an in-class defined, non-deleted, constexpr default constructor
7      unique_ptr() = default;
8
9      constexpr explicit unique_ptr(T *v) : _v(v) {}
10
11     unique_ptr(const unique_ptr &) = delete;
12     unique_ptr &operator=(const unique_ptr &) = delete;
13
14     unique_ptr(unique_ptr &&) = relocates;
15     unique_ptr &operator=(unique_ptr &&o) noexcept
16     {
17         delete _v;
18         _v = o._v;
19         o._v = nullptr;
20         return *this;
21     }
22     ~unique_ptr()
23     {

```

```

24     delete _v;        // No side effects when _v == nullptr
25     _v = nullptr;
26 }
27
28 T &operator*() noexcept { return *_v; }
29 };

```

The default constructor is not deleted, constexpr and defined in-class, and it sets the single, trivially copyable, member data `_v` to `nullptr`. No base classes nor member variables are neither trivially copyable nor move relocating, so the application of `= relocates` does not cause a compile time diagnostic.

The destructor, when called on a default constructed instance, will be reduced by the optimiser to a trivial destructor (`operator delete` does nothing when fed a null pointer, and setting a null pointer to a null pointer leaves the object with exactly the same memory representation as a default constructed instance).

We shall compile this small program and see how it looks before and after the attribute has been applied:

```

1  extern unique_ptr<int> __attribute__((noinline)) boo()
2  {
3      return unique_ptr<int>(new int);
4  }
5
6  extern unique_ptr<int> __attribute__((noinline)) foo()
7  {
8      auto a = boo();
9      *a += *boo();
10     return a;
11 }
12
13 int main()
14 {
15     auto a = foo();
16     return 0;
17 }

```

3.1.1 With current compilers, without `= relocates`:

On current C++ compilers⁴, the program will generate the following x64 assembler:

```

1  boo():
2      push rbx
3      mov rbx, rdi
4      mov edi, 4
5      call operator new(unsigned long)
6      mov QWORD PTR [rbx], rax
7      mov rax, rbx
8      pop rbx
9      ret

```

⁴GCC 8 with `-O2` on.

As `unique_ptr` is not a trivially copyable type, the compiler is forced to use stack storage to return the `unique_ptr`. The caller passes in where it wants the return stored in `rdi`, which is saved into `rbx`. It allocates four bytes (`edi`) for the `int` using `operator new`, and places the pointer to the allocated memory into the eight bytes pointed to by `rbx`. It returns the pointer to the pointer to the allocated `int` via `rax`.

```

1  foo():
2      push rbp
3      push rbx
4      mov rbx, rdi
5      sub rsp, 24
6      call boo()
7      lea rdi, [rsp+8]
8      call boo()
9      mov rdi, QWORD PTR [rsp+8]
10     mov rax, QWORD PTR [rbx]
11     mov esi, 4
12     mov edx, DWORD PTR [rdi]
13     add DWORD PTR [rax], edx
14     call operator delete(void*, unsigned long)
15     add rsp, 24
16     mov rax, rbx
17     pop rbx
18     pop rbp
19     ret
20     mov rbp, rax
21     jmp .L5
22 foo() [clone .cold.1]:
23 .L5:
24     mov rdi, QWORD PTR [rbx]
25     mov esi, 4
26     call operator delete(void*, unsigned long)
27     mov rdi, rbp
28     call _Unwind_Resume

```

We firstly allocate 24 bytes on the stack frame (`rsp`) for the two `unique_ptr`s, calling `boo()` twice to fill each in. We load the two pointers to the two `int`'s from the two `unique_ptr`s (`rdi`, `rax`), dereference that into the allocated `int` for one (`edx`) and add it directly to the memory pointed to by `rax`. We call `operator delete` on the added-from `unique_ptr`, returning the added-to `unique_ptr`.

```

1  main:
2      sub rsp, 24
3      lea rdi, [rsp+8]
4      call foo()
5      mov rdi, QWORD PTR [rsp+8]
6      mov esi, 4
7      call operator delete(void*, unsigned long)
8      xor eax, eax
9      add rsp, 24
10     ret

```

After reserving space for the returned `unique_ptr` filled in by calling `foo()`, `main()` loads the pointer to the allocated memory returned by `foo()`, and calls `operator delete` on it. This is `unique_ptr`'s destructor correctly firing on destruction of the `unique_ptr`.

3.1.2 With the proposed = relocates:

Now let us look at the x64 assembler which would be generated instead if this proposal were in place:

```
1 boo():
2     mov edi, 4
3     jmp operator new(unsigned long) # TAILCALL
```

The compiler now knows that unique ptrs can be stored in registers because moves relocate. Knowing this, it optimises out entirely the use of stack to transfer instances of unique ptrs, and thus simply returns in `rax` a naked pointer to a four byte allocation for the `int`. In other words, the `unique_ptr` implementation is entirely eliminated, just its data member an `int*` remains!

```
1 foo():
2     push rbx
3     call boo()
4     mov rbx, rax
5     call boo()
6     mov esi, 4
7     mov edx, DWORD PTR [rax]
8     add DWORD PTR [rbx], edx
9     mov rdi, rax
10    call operator delete(void*, unsigned long)
11    mov rax, rbx
12    pop rbx
13    ret
```

`foo()` has become rather simpler, too. `boo()` returns the allocated `int` directly in `rax`, so now the compiler can simply dereference one of them once, add it to the memory pointed to by the other. No more double dereferencing!

The first unique ptr is destructed, and we return the second unique ptr in `rax`.

```
1 main:
2     call foo()
3     mov esi, 4
4     mov rdi, rax
5     call operator delete(void*, unsigned long)
6     xor eax, eax
7     ret
```

`main()` has become almost trivially simple. We call `foo()`, and delete the pointer it returns before returning zero from `main()`.

3.1.3 How do you know that the code in the second example is feasibly generatable by a compiler?

The second example is not hand written. I actually created two unique ptr implementations, one trivially copyable and one the above, and used forced casting to introduce trivially copyable semantics at the correct points. The code you see above was actually generated by a mixture of clang trunk and GCC trunk, using those forced type castings to mimic the proposed semantics.

Upon reviewing this paper, Richard Smith suggested that applying the `[[clang::trivial_abi]]` attribute might result in similar elision of `unique_ptr`. This was tested and found to be true.

3.2 So what?

Those of you who are used to counting assembler opcode latency will immediately see that the second edition is many times faster than the first edition *because it depends on memory much less*. Even though reads and writes to the stack are probably L1 cache fast, any read or write to memory is far slower than CPU registers, typically a maximum of one operation per cycle with a latency of as much as three cycles. CPU registers typically can issue four operations per cycle, with between a zero and one cycle latency. If you add up the CPU cycles in the two examples above, excluding operators `new` and `delete`, you will find the second example is several times faster with a fully warmed L1 cache.

What is hard to describe to the uninitiated is how well this microoptimisation aggregates over a whole program. If you make all the types in your program trivially copyable, you will see across the board performance improvements with especial gain in *performance consistency*.

This is why SG14, the low latency study group, would really like for WG21 to standardise relocation so a greater range of types can be brought under maximum optimisation, including [\[P0709\]](#) *Zero-overhead deterministic exceptions: Throwing values* and [\[P1031\]](#) *Low level file i/o library*, both of which would make great use of move relocates.

4 Design decisions, guidelines and rationale

Previous work in this area has tended towards the complex. This proposal proposes the barest of essentials for a limited subset of address relocatable types in the hope that the committee will be able to get this passed.

5 Technical specifications

No Technical Specifications are involved in this proposal.

6 Acknowledgements

Thanks to Richard Smith for his extensive thoughts on the feasibility, and best formulation, of this proposal.

Thanks to Arthur O'Dwyer for his feedback from his alternative relocatable proposal.

Thanks to Nicol Bolas for quite extensive feedback and commentary, and to Alberto Barbati for feedback helping me reduce the size of the proposal still further.

7 References

- [N4034] Pablo Halpern,
Destructive Move
<https://wg21.link/N4034>
- [P0023] Denis Bider,
Relocator: Efficiently moving objects
<https://wg21.link/P0023>
- [P0709] Herb Sutter,
Zero-overhead deterministic exceptions: Throwing values
<https://wg21.link/P0709>
- [P0784] Dionne, Smith, Ramms and Vandevorde,
Standard containers and constexpr
<https://wg21.link/P0784>
- [P1028] Douglas, Niall
SG14 status_code and standard error object for P0709 Zero-overhead deterministic exceptions
<https://wg21.link/P1028>
- [P1031] Douglas, Niall
Low level file i/o library
<https://wg21.link/P1031>
- [P1144] Arthur O'Dwyer, Mingxin Wang
Object relocation in terms of move plus destroy
<https://wg21.link/P1144>
- [P1631] Douglas, Niall and Steagall, Bob
Object detachment and attachment
<https://wg21.link/P1631>